Problem 1: Batch processing

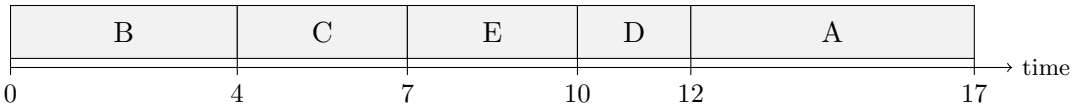
I use the notation from the slides:

$$T_{\text{completion}} = T_{\text{end}} - T_{\text{insertion}}, \qquad T_{\text{wait}} = T_{\text{completion}} - \text{CPU time}.$$

Here $T_{\text{completion}}$ is the turnaround time. The penalty rate is $T_{\text{completion}}/\text{CPU time}$.

1.1 SJF (non-preemptive)

Whenever the CPU becomes free, I choose the ready task with the shortest CPU time. At $t = 0$, only A and B are ready, so B runs first. C is ready when B finishes, and E is ready when C finishes. D then runs before A because it is shorter. This gives the order B, C, E, D, A.



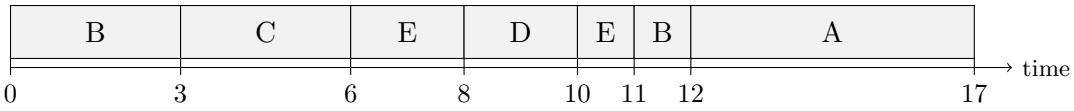
Task	CPU time	Arrival	Finish	Turnaround	Waiting	Penalty rate
A	5	0	17	$17 - 0 = 17$	$17 - 5 = 12$	$17/5 = 3.4$
B	4	0	4	$4 - 0 = 4$	$4 - 4 = 0$	$4/4 = 1$
C	3	3	7	$7 - 3 = 4$	$4 - 3 = 1$	$4/3 \approx 1.33$
D	2	8	12	$12 - 8 = 4$	$4 - 2 = 2$	$4/2 = 2$
E	3	5	10	$10 - 5 = 5$	$5 - 3 = 2$	$5/3 \approx 1.67$
Average				$34/5 = 6.8$	$17/5 = 3.4$	$9.4/5 = 1.88$

The average waiting time is 3.4. SJF favors short tasks, but it needs an estimate of their CPU times. There is a risk of starvation (famine): A is ready from $t = 0$ but waits until $t = 12$ because all the shorter tasks go first. If short tasks kept arriving, a long task could keep waiting indefinitely.

1.2 PSJN (Preemptive Shortest Job Next)

For this part, I compare the tasks by their *total* CPU time, rather than the remaining time used by SRTF. A new task preempts the running task if its total CPU time is shorter. I assume equal CPU times do not cause preemption, so E does not interrupt C at $t = 5$.

- $t = 0$: A(5) and B(4) are ready, B runs.
- $t = 3$: C(3) is inserted. $3 < 4$, so C preempts B (B has done 3, 1 unit left).
- $t = 5$: E(3) is inserted, same CPU time as C, so no preemption, C keeps running.
- $t = 6$: C ends. Ready: A(5), B(4), E(3). E runs.
- $t = 8$: D(2) is inserted, $2 < 3$, D preempts E (E has 1 unit left).
- $t = 10$: D ends. E gets its last unit on $[10, 11]$.
- $t = 11$: E ends. B finishes its last unit on $[11, 12]$, then A runs on $[12, 17]$.



Task	CPU time	Arrival	Finish	Turnaround	Waiting	Penalty rate
A	5	0	17	$17-0=17$	$17-5=12$	$17/5=3.4$
B	4	0	12	$12-0=12$	$12-4=8$	$12/4=3$
C	3	3	6	$6-3=3$	$3-3=0$	$3/3=1$
D	2	8	10	$10-8=2$	$2-2=0$	$2/2=1$
E	3	5	11	$11-5=6$	$6-3=3$	$6/3=2$
Average				$40/5=8$	$23/5=4.6$	$10.4/5=2.08$

The average turnaround time is 8, compared with 6.8 in 1.1. Both B and E are interrupted with only 1 unit left, since this rule ignores how much CPU time they have already used. Starvation is still possible: repeated arrivals of shorter tasks can delay a long task even after it has started running.

Problem 2: Timesharing

2.1 Round-robin, $Q = 100$ ms

The first seven rows are given in the problem sheet; I added the rows below the middle line. T2 requests I/O after every 10 ms of CPU time, and T4 after every 20 ms. Their I/Os take 250 ms and 180 ms respectively and can overlap on the two disks. I include the final I/O before each task terminates.

I follow the queue rule in the given rows: r means ready and b means blocked. When I/O finishes, the task becomes ready at its current queue position and does not interrupt the running task. Blocked tasks are skipped when choosing the next task; if the CPU is idle, a task that becomes ready can run immediately.

Date	Event(s)	Elected task	State of the queue	End of task
0	load T1	T1	1r 2r 3r 4r	
100	switch	T2	2r 3r 4r 1r	
110	I/O T2, switch	T3	3r 4r 1r 2b	
210	switch	T4	4r 1r 2b 3r	
230	I/O T4, switch	T1	1r 2b 3r 4b	
330	switch	T3	2b 3r 4b 1r	
360	end of I/O T2	T3	2r 3r 4b 1r	
410	end of I/O T4	T3	2r 3r 4r 1r	
430	end of T3, switch	T2	2r 4r 1r	T3
440	I/O T2, switch	T4	4r 1r 2b	

Date	Event(s)	Elected task	State of the queue	End of task
460	I/O T4, switch	T1	1r 2b 4b	
560	end of T1	– (idle)	2b 4b	T1
640	end of I/O T4, end of T4	– (idle)	2b	T4
690	end of I/O T2, load T2	T2	2r	
700	I/O T2	– (idle)	2b	
950	end of I/O T2, end of T2	– (idle)	–	T2

As a check, T1 runs for $3 \times 100 = 300$ ms, T3 for $2 \times 100 = 200$ ms, T2 for $3 \times 10 = 30$ ms, and T4 for $2 \times 20 = 40$ ms. T4 finishes after its last I/O on $[460, 640]$, and T2 after its last I/O on $[700, 950]$. The CPU is idle on $[560, 690]$ and from 700 until all tasks finish.

All tasks arrive at $t = 0$, so their turnaround times are their finish times: T1 = 560, T2 = 950, T3 = 430, and T4 = 640 ms.

2.2 Advantages, disadvantages, and starvation

Round-robin is simple and gives each ready task a turn without needing its CPU time in advance. A short task can run after one quantum of a long task instead of waiting for the whole task to finish, which helps response time.

However, it does not distinguish urgent tasks from other tasks. I/O-bound tasks can also wait a long time between short CPU bursts. Here T4 uses only 40 ms of CPU but waits $210 + 30 = 240$ ms in the ready queue. T2 finishes last at 950 ms, giving a penalty rate of $950/30 \approx 32$, although 750 ms comes from its own I/O. The choice of Q matters: a large quantum makes RR behave like FCFS, while a very small quantum causes frequent context switches.

There is no starvation for a task that stays ready. After each turn, the running task finishes or moves to the back of the queue, so waiting ready tasks move closer to the front. With n tasks in the queue, a ready task waits at most $(n - 1)Q$, ignoring context-switch overhead. Time blocked on I/O is separate from this waiting time.

Problem 3: Timesharing with static priorities

3.1 Round-robin on a multilevel queue

Priority 3 is the highest. Initially, level 3 contains T2 then T4, level 1 contains T3 then T5, and level 0 contains T1. I use RR with $Q = 100$ ms within each level and choose the highest level with a ready task.

As in 2.1, I assume an I/O completion does not interrupt the running task. The next task is chosen when a quantum ends, a task requests I/O, or a task terminates. Level 2 stays empty, so I leave it out of the table.

Date	Event(s)	Elected task	State of the queues			End of task
			level 3	level 1	level 0	
0	load T2	T2	2r 4r	3r 5r	1r	
10	I/O T2, switch	T4	4r 2b	3r 5r	1r	
30	I/O T4, switch	T3	2b 4b	3r 5r	1r	
130	switch	T5	2b 4b	5r 3r	1r	
210	end of I/O T4	T5	2b 4r	5r 3r	1r	
230	switch	T4	2b 4r	3r 5r	1r	
250	I/O T4, switch	T3	2b 4b	3r 5r	1r	
260	end of I/O T2	T3	2r 4b	3r 5r	1r	
350	end of T3, switch	T2	2r 4b	5r	1r	T3
360	I/O T2, switch	T5	4b 2b	5r	1r	
430	end of I/O T4, end of T4	T5	2b	5r	1r	T4
460	switch (T5 re-elected)	T5	2b	5r	1r	
560	end of T5, switch	T1	2b	–	1r	T5
610	end of I/O T2	T1	2r	–	1r	
660	switch	T2	2r	–	1r	
670	I/O T2, switch	T1	2b	–	1r	
770	switch (T1 re-elected)	T1	2b	–	1r	
870	end of T1	– (idle)	2b	–	–	T1
920	end of I/O T2, end of T2	– (idle)	–	–	–	T2

The total CPU time is $300 + 30 + 200 + 40 + 300 = 870$ ms. There is no idle time before $t = 870$, when T1 finishes; T2's final I/O ends later. The turnaround times are T1 = 870, T2 = 920, T3 = 350, T4 = 430, and T5 = 560 ms.

T2 and T4 get the CPU at the next scheduling point after their I/O finishes. T4 now finishes at 430 ms, compared with 640 ms in 2.1. T1 has the lowest priority and does not get its first turn until $t = 560$.

3.2 Advantages, disadvantages, and starvation

Static priorities let important tasks run before less important ones. Giving T2 and T4 high priority also lets their I/O overlap with other tasks' CPU work. RR keeps tasks with the same priority sharing the CPU fairly.

The drawback is that a poorly chosen priority stays fixed. Low-priority tasks can wait a long time, as T1 does here. Priority inversion is also possible if a low-priority task holds a resource needed by a high-priority task while medium-priority tasks prevent it from running.

Starvation is possible because a lower level only runs when all higher levels have no ready tasks. If higher-priority tasks keep the CPU busy, T1 may never run. Ageing, or reserving a share of CPU time for each level, would prevent this.

Problem 4: Timesharing with dynamic priorities

4.1 Multilevel feedback queue, $Q = 100$ ms, $0 \le d \le 900$

Since the number of queues is not specified, I keep the four levels from 3.1. New tasks enter level 3. A task that uses a full quantum moves to the back of the next lower level; at level 0 it stays at level 0. I assume new arrivals do not interrupt a running quantum.

T1.x arrives at $150x$ and needs 15 ms, T2.y arrives at $300y$ and needs 200 ms, and T3.0 arrives at 0 and needs 1000 ms. In the queue columns, $x.y$ means T$x.y$. All tasks are ready, so I omit the r/b marks.

Date	Event(s)	Elected task	State of the queues (all tasks ready)				End of task
			level 3	level 2	level 1	level 0	
0	insert T1.0, T2.0, T3.0; load T1.0	T1.0	1.0 2.0 3.0	–	–	–	
15	end of T1.0, switch	T2.0	2.0 3.0	–	–	–	T1.0
115	switch (T2.0 → level 2)	T3.0	3.0	2.0	–	–	
150	insert T1.1	T3.0	3.0 1.1	2.0	–	–	
215	switch (T3.0 → level 2)	T1.1	1.1	2.0 3.0	–	–	
230	end of T1.1, switch	T2.0	–	2.0 3.0	–	–	T1.1
300	insert T1.2, T2.1	T2.0	1.2 2.1	2.0 3.0	–	–	
330	end of T2.0, switch	T1.2	1.2 2.1	3.0	–	–	T2.0
345	end of T1.2, switch	T2.1	2.1	3.0	–	–	T1.2
445	switch (T2.1 → level 2)	T3.0	–	3.0 2.1	–	–	
450	insert T1.3	T3.0	1.3	3.0 2.1	–	–	
545	switch (T3.0 → level 1)	T1.3	1.3	2.1	3.0	–	
560	end of T1.3, switch	T2.1	–	2.1	3.0	–	T1.3
600	insert T1.4, T2.2	T2.1	1.4 2.2	2.1	3.0	–	
660	end of T2.1, switch	T1.4	1.4 2.2	–	3.0	–	T2.1
675	end of T1.4, switch	T2.2	2.2	–	3.0	–	T1.4
750	insert T1.5	T2.2	2.2 1.5	–	3.0	–	
775	switch (T2.2 → level 2)	T1.5	1.5	2.2	3.0	–	
790	end of T1.5, switch	T2.2	–	2.2	3.0	–	T1.5
890	end of T2.2, switch	T3.0	–	–	3.0	–	T2.2
900	insert T1.6, T2.3	T3.0	1.6 2.3	–	3.0	–	

At $t = 900$ T3.0 is in its third quantum, $[890, 990]$; it had received 200 ms of CPU ($[115, 215]$ and $[445, 545]$) when it was elected at 890. After this quantum it drops to level 0 with $1000 - 300 = 700$ ms left.

4.2 Termination, starvation, and a modified algorithm

Will T3.0 terminate? Yes, under these assumptions. The periodic tasks use

$$\frac{15}{150} + \frac{200}{300} = 10\% + 66.7\% = 76.7\%$$

of the CPU, leaving about 23.3% for T3.0. In this schedule, the higher levels become empty from time to time, allowing T3.0 to run.

Continuing the table beyond 900, I get the later CPU intervals

$$[1450, 1550], \quad [1780, 1880], \quad [2340, 2440], \quad [2670, 2770],$$
$$[3230, 3330], \quad [3560, 3660], \quad [3890, 3990].$$

Thus T3.0 finishes at $t = 3990$, with turnaround time 3990 ms and penalty rate $3990/1000 = 3.99$. For simultaneous arrivals and task or quantum endings, I add the new tasks to their queues before choosing the next task.

Is termination guaranteed in general? No. If higher levels always have ready tasks, a task in a lower level can starve. For example, if T2 needs 200 ms every 200 ms instead of every 300 ms, the new tasks need $10\% + 100\% = 110\%$ of the CPU. T2 tasks then accumulate in levels 3 and 2, and T3.0 stops getting turns after it reaches level 1.

Proposed algorithm. To satisfy (a), I would promote tasks that use their whole quantum. To satisfy (b), I would add ageing. The rules are:

- *Queues:* N levels, from $N-1$ (highest) to 0. For example, use $N = 4$ and $Q = 100$ ms.
- *Scheduling:* RR within each level; choose the highest level with a ready task and skip blocked tasks.
- *Promotion:* after using a full quantum at level k, a task joins the back of level $\min(k+1,\ N-1)$.
- *Ageing:* after waiting ready for W ms without running, a task below the top level moves up one level and its waiting counter resets. For example, take $W = 500$ ms. Tasks already at the top keep their queue positions.
- *Demotion:* a task that requests I/O before its quantum ends joins the back of level $\max(k-1,\ 0)$.
- *New tasks:* start at a middle level, such as level 1 or 2 when $N = 4$.

A task that stays ready without running reaches the top level within $(N-1)W$. Once there, tasks arriving or promoted later go behind it. If there are $n-1$ tasks ahead of it, it waits at most nQ, including the current quantum. The total bound is therefore $(N-1)W + nQ$, ignoring context-switch overhead. With $N = 4$ and $W = 500$ ms, the ageing part is at most 1.5 s. This also prevents I/O-bound tasks from starving after repeated demotions.